SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Testing Strategies for Reliable Embedding-Driven ML Systems: A Case Study on Biocentral Services

Author:	Deniz Özturk
Examiner:	Sebastian Franz
Supervisor:	Advisor
Submission Date:	Submission date

I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, Submission date Deniz Özturk

Transparency in the Use of AI Tools

In preparing this thesis, I utilized Grammarly to correct grammatical errors, improve sentence structure, and enhance overall writing style. I used ChatGPT to generate initial drafts, expand on ideas, providing valuable suggestions and examples. I have carefully checked all texts created with these tools to ensure that they are correct and make sense.

Acknowledgments

Abstract

Protein language models (PLMs) such as ESM2 and ProtT5 are increasingly integrated into production bioinformatics platforms, yet systematic approaches to testing these embedding-driven systems remain scarce. The absence of ground-truth oracles, GPU-induced non-determinism, and high computational cost make traditional software testing techniques insufficient.

This thesis presents a comprehensive, multi-layer testing strategy for PLM-based services, developed and evaluated as a case study on the biocentral server—an open-source platform for embedding-based protein analysis. The strategy comprises five layers: unit tests with mocked dependencies, integration tests against the live server, property-based oracle tests verifying mathematical invariants, performance benchmarks, and metamorphic experiments validating PLM embedding behavior at scale.

Key contributions include (1) a deterministic, GPU-free testing infrastructure (*FixedEmbedder*) that enables the full integration test suite to run in CI without GPU resources, (2) eight metamorphic experiments—including progressive X-masking, reversed sequence sensitivity, and amino acid mutation sensitivity—validated on 1,750 real protein sequences from UniRef50, (3) a reusable oracle testing framework with statistical utilities for robust invariant checking, and (4) mutation testing evaluation of the test suite's fault-detection capability.

The evaluation quantifies the trade-off between CI execution time and test coverage across all layers, providing concrete recommendations for allocating testing budgets in embedding-driven ML services. Results demonstrate that integration tests with the FixedEmbedder offer the strongest cost-effectiveness ratio, while metamorphic experiments provide unique behavioral insights into PLM properties that no other test layer captures.

Contents

Transparency in the Use of AI Tools **3**

Acknowledgments **4**

Abstract **5**

1 Introduction **1**
1.1 Motivation 1
1.2 Problem Statement 2
1.3 Research Question 3
1.4 Contributions 3
1.5 Thesis Outline 4

2 Background **5**
2.1 Protein Language Models 5
2.2 The Biocentral Platform 5
2.3 Software Testing Fundamentals 6
2.4 Challenges of Testing ML Systems 7
2.5 Metamorphic Testing 7
2.6 Property-Based and Oracle Testing 8

3 Related Work **10**
3.1 Testing Machine Learning Systems 10
3.2 Metamorphic Testing for Deep Learning 10
3.3 Testing Scientific and Bioinformatics Software 11
3.4 Property-Based Testing for Data Pipelines 11
3.5 Research Gap 11

4 Approach **13**
4.1 Overall Test Architecture 13
4.2 Test Infrastructure 13
4.2.1 Canonical Test Dataset 13
4.2.2 FixedEmbedder 14

4.2.3 Integration Test Infrastructure . . . 14
4.3 Unit Tests . . . 15
4.4 Integration Tests . . . 15
4.5 Property-Based Oracle Tests . . . 16
4.6 Metamorphic Experiments . . . 17
4.6.1 Idempotency . . . 17
4.6.2 Batch Invariance . . . 17
4.6.3 Progressive X-Masking . . . 18
4.6.4 Reversed Sequence . . . 18
4.6.5 Amino Acid Mutation Sensitivity . . . 18
4.6.6 Random Baseline . . . 19
4.6.7 Projection Determinism . . . 19
4.6.8 Training Determinism . . . 19
4.7 Performance Tests . . . 19
4.8 Mutation Testing . . . 20

5 Evaluation **21**
5.1 Experimental Setup . . . 21
5.2 Test Coverage and Mutation Testing Results . . . 21
5.3 CI Execution Time Analysis . . . 22
5.4 Metamorphic Experiment Results . . . 22
5.4.1 X-Masking Results . . . 22
5.4.2 Reversed Sequence Results . . . 23
5.4.3 AA Mutation Sensitivity Results . . . 23
5.5 Oracle Test Findings . . . 24
5.6 Performance Results . . . 24
5.7 Discussion of Results . . . 24

6 Discussion **26**
6.1 Trade-Off Summary . . . 26
6.2 Generalizability . . . 26
6.3 Limitations . . . 27
6.4 Threats to Validity . . . 27

7 Conclusion **29**
7.1 Summary of Contributions . . . 29
7.2 Key Findings . . . 29
7.3 Future Work . . . 30

Abbreviations **31**

List of Figures **32**

List of Tables **33**

Bibliography **34**

1 Introduction

1.1 Motivation

Protein Language Model (PLM)-based services have become a central component in computational biology research. Models such as ESM2 and ProtT5 encode protein sequences into high-dimensional embedding vectors that capture structural and functional properties learned from large-scale pre-training on protein databases [Eln+21]. These embeddings enable a broad range of downstream analyses, including secondary structure prediction, subcellular localization, and variant effect estimation, without requiring explicit feature engineering. As a result, embedding-driven systems are increasingly deployed as production services that expose PLM capabilities through Application Programming Interface (API) endpoints.

Biocentral is one such system: an open-source, production-grade platform for embedding-based protein analysis [Fra+25]. Its server component provides nine API modules spanning embedding computation, prediction serving, dimensionality reduction, custom model training, and Bayesian optimization. The platform relies on Graphics Processing Unit (GPU)-accelerated inference via NVIDIA Triton Inference Server, asynchronous task pipelines backed by Redis, and a PostgreSQL database for embedding caching. This architecture enables scalable protein analysis but introduces substantial complexity in the software stack that must be validated through systematic testing.

Traditional software testing relies on two fundamental assumptions: that program outputs are deterministic for a given input, and that expected outputs are known in advance. Neither assumption holds for Machine Learning (ML) embedding systems. PLM embeddings lack a ground-truth oracle; there is no reference output against which a given embedding vector can be verified as correct. GPU-based floating-point arithmetic introduces non-determinism across hardware configurations and driver versions, meaning that identical inputs may produce numerically different outputs across runs [Cha+23]. Furthermore, the computational cost of loading and executing large transformer models renders exhaustive test execution impractical within typical Continuous Integration (CI) time budgets. These characteristics—the absence of an oracle, hardware-induced non-determinism, and prohibitive compute requirements—distinguish ML embedding services from conventional software systems and demand

testing approaches specifically adapted to these constraints.

Despite the growing deployment of PLM-based services, the problem of testing such systems remains under-explored relative to the attention devoted to model development and training. Research on CI practices in ML projects indicates that testing is frequently deprioritized in favor of model performance optimization [Ber+25; Rzi+25]. This imbalance is problematic: deployment-grade ML services must satisfy not only model-level accuracy requirements but also system-level correctness guarantees covering API contracts, data pipeline integrity, caching consistency, and inference reproducibility.

1.2 Problem Statement

The Biocentral server integrates nine API modules, GPU-accelerated inference through Triton, asynchronous task pipelines with worker processes, and database-backed embedding caching into a single deployable system. Each of these components introduces distinct failure modes: API endpoints may violate their contracts, the inference server may produce inconsistent results across model versions, task pipelines may lose or corrupt intermediate state, and the caching layer may serve stale or incorrectly keyed embeddings. Ensuring correctness across this heterogeneous stack requires a test strategy that addresses each layer individually while also validating the interactions between layers.

Conventional unit and integration tests are necessary but insufficient for this setting. Unit tests verify isolated functions but cannot detect faults that arise from component interactions or from the statistical nature of ML outputs. Integration tests validate system-level workflows but depend on infrastructure availability and do not address the oracle problem inherent in embedding computation. Property-based and metamorphic testing techniques offer a principled approach to validating ML behavior in the absence of ground truth—for example, by asserting that semantically similar inputs produce similar embeddings, or that known sequence transformations induce predictable changes in embedding space. However, these techniques require executing real PLM inference, which is computationally expensive and typically demands GPU hardware.

This creates a fundamental tension between test thoroughness and CI execution time. A comprehensive test suite that includes property-based and metamorphic tests over large protein datasets may require hours of GPU computation, rendering it impractical for execution on every commit. Conversely, a fast test suite restricted to unit and lightweight integration tests may fail to detect faults specific to ML inference behavior. The central problem addressed in this thesis is therefore the design and evaluation of a multi-layer test strategy for embedding-driven ML services that balances fault-detection

capability against the practical constraints of CI pipelines.

1.3 Research Question

The preceding observations motivate the following research question:

> Research Question (RQ): *What is the trade-off between CI execution time and fault-detection capability when applying property-based and metamorphic testing to embedding-driven ML services?*

This question is decomposed into two sub-questions. First, which test layers—unit, integration, property-based, or metamorphic—provide the highest fault-detection value per unit of CI time? Second, can GPU-free deterministic embedders replace real PLM models in CI pipelines without reducing the structural coverage achieved by the test suite?

1.4 Contributions

This thesis makes the following contributions:

1. A multi-layer test pyramid tailored to embedding-driven ML services, organized into five layers: unit tests, integration tests, property-based and oracle tests, performance benchmarks, and metamorphic experiments.
2. `FixedEmbedder`, a SHA-256-based deterministic embedder that produces reproducible embedding vectors from protein sequences without requiring GPU hardware or neural network inference. This component enables the full CI pipeline to execute on commodity hardware.
3. Eight metamorphic experiments that validate PLM embedding behavior under controlled sequence transformations, including unknown-token masking, sequence reversal, and single-residue mutation sensitivity, evaluated at UniRef50 scale.
4. An oracle test pattern supported by statistical utilities for confidence interval estimation, effect size computation, and significance threshold evaluation, enabling quantitative assertions on ML outputs.
5. A mutation testing evaluation using `mutmut` that quantifies the fault-detection capability of the unit test suite.

6. A quantitative analysis of the trade-off between CI execution time and fault-detection coverage across the test layers, providing empirical guidance for test portfolio selection.

1.5 Thesis Outline

The remainder of this thesis is organized as follows.

Chapter 2 introduces the foundational concepts underlying this work, including PLM architectures and their embedding representations, the Biocentral platform and its server-side components, and software testing fundamentals from the test pyramid to property-based and metamorphic testing.

Chapter 3 surveys related work on testing ML systems, covering metamorphic testing for deep learning, testing practices in scientific software, and property-based testing approaches for data pipelines.

Chapter 4 presents the test architecture, infrastructure design, and experimental methodology. This includes the five-layer test pyramid, the `FixedEmbedder` design, the canonical test dataset, integration test infrastructure, and the metamorphic experiment specifications.

Chapter 5 reports the evaluation results, including code coverage and mutation testing metrics, CI execution time measurements per test layer, and the quantitative trade-off analysis between testing thoroughness and pipeline speed.

Chapter 6 interprets the findings, assesses generalizability to other ML service architectures, acknowledges limitations of the approach, and identifies threats to internal, external, and construct validity.

Chapter 7 summarizes the contributions and outlines directions for future work.

2 Background

2.1 Protein Language Models

- Proteins as sequences of amino acids; 20 standard residues + special tokens (X, B, Z, U, O)
- Language model paradigm: treat protein sequences like natural language sentences
- Transformer architecture applied to proteins (self-attention, positional encoding)
- Pre-training objective: masked language modeling on large protein databases (UniRef, BFD)
- Key models: ESM2 (Meta, 8M to 15B parameters), ProtT5 (Elnaggar et al.), ProstT5
- Embeddings: per-residue (sequence length $\times$ hidden dim) vs. pooled (single vector per protein via mean reduction)
- Downstream tasks enabled by embeddings: secondary structure prediction, subcellular localization, binding site prediction, disorder prediction, conservation, variant effects
- Baseline encodings for comparison: one-hot encoding, BLOSUM62 substitution matrix

2.2 The Biocentral Platform

- Overview: open-source platform for embedding-based protein analysis (Franz et al., JMB 2026)
- Client-server architecture: Flutter desktop client + FastAPI backend
- Server components: FastAPI web framework, Redis task queue (rq), PostgreSQL embedding database, SeaweedFS file storage, NVIDIA Triton Inference Server for GPU inference

- Docker Compose orchestration: 7+ services (server, workers, Redis, PostgreSQL, SeaweedFS, Triton, Prometheus)
- Async task pattern: POST returns `task_id` → client polls `/task_status/{task_id}` until completion
- Worker model: 4 RQ worker processes with priority queues (high, default, low)
- API modules overview (each under `/api/v1/`):
  - Embeddings: compute PLM embeddings, cache in PostgreSQL, HDF5 import/export
  - Predictions: 9 pre-trained ONNX models (TMbed, Seth, BindEmbed, VespaG, etc.)
  - Projections: PCA, UMAP, t-SNE via ProtSpace library
  - Custom Models: train/evaluate custom ML models via biotrainer
  - Active Learning / Bayesian Optimization: GP and FNN-based iterative protein engineering
  - PLM Eval: automated PLM benchmarking
  - PPI: protein-protein interaction dataset analysis
  - Proteins: taxonomy lookup
- Middleware: CORS, rate limiting (FastAPI-Limiter), body size limits (200 MB), user management via request headers
- Figure idea: architecture diagram showing client → server → workers → Triton/DB

2.3 Software Testing Fundamentals

- The test pyramid: unit tests (fast, isolated, mocked) → integration tests (component interaction, live services) → system/E2E tests (full stack)
- Unit testing: test isolation via mocking/stubbing, fast feedback, high coverage of edge cases
- Integration testing: verify component interactions, database queries, API contracts, async flows

- Test-driven development and continuous integration (CI): run tests on every commit, fast feedback loops
- Key concepts: test fixtures, parametrization, markers, session-scoped setup
- Pytest framework: markers (`@pytest.mark.integration`, `@pytest.mark.slow`), fixtures, conftest.py hierarchy
- Code coverage vs. fault detection: coverage measures executed lines, not correctness
- Mutation testing: inject syntactic faults (mutants), check if tests detect them (mutation score), mutmut tool

2.4 Challenges of Testing ML Systems

- The oracle problem: no known "correct" output for a given protein embedding — cannot write `assert embedding == expected`
- Non-determinism: GPU floating-point operations are non-associative; batch ordering, cuDNN algorithm selection, and parallelism cause bit-level variation
- Data dependency: model behavior depends on training data, not just code — traditional unit tests miss data-induced bugs
- Computational cost: embedding a single protein can take seconds on GPU; full test suites with real models are prohibitively slow for CI
- Model opacity: transformer internals are not interpretable — testing must focus on behavioral properties rather than internal state
- Infrastructure complexity: multi-service deployments (server + workers + GPU + DB + file storage) create integration failure modes absent in monolithic software
- Versioning: model weights, tokenizers, and inference frameworks (ONNX, Triton) must all be version-consistent

2.5 Metamorphic Testing

- Definition: testing technique that verifies *relations* between inputs and outputs across multiple executions, rather than checking individual outputs against expected values

- Metamorphic relations (MRs): if input is transformed in a specific way, the output should change (or not change) in a predictable way
- Advantage: circumvents the oracle problem — does not require knowing the "correct" output
- Classic example: $\sin(x) = \sin(\pi - x)$ — can verify without knowing $\sin(x)$ for any specific x
- Application to ML: e.g., adding noise to input should not drastically change classification; rotating an image should not change object detection
- Application to PLM embeddings: replacing residues with X (unknown token) should cause monotonically increasing divergence; reversing sequence order should change embedding (positional encoding sensitivity); specific AA substitutions should cause predictable sensitivity patterns
- Distinction from property-based testing: MRs relate multiple executions; properties constrain single executions
- Key papers: Chen et al. (1998) original proposal; Segura et al. (2016) survey; applications in ML by Xie et al.

2.6 Property-Based and Oracle Testing

- Property-based testing: specify invariants that must hold for all (or randomly generated) inputs, rather than specific input-output pairs
- Tools: Hypothesis (Python), QuickCheck (Haskell) — generate random inputs, check properties
- Oracle pattern: encapsulate a reusable invariant check in a class with a `check()` method returning structured pass/fail + metrics
- Examples of oracles for embedding systems:
  - Determinism: same input $\rightarrow$ same output (within tolerance for GPU)
  - Batch invariance: embedding unchanged regardless of batch composition
  - Output validity: probabilities in $[0, 1]$, sum to 1 (classification); finite values (regression)
  - Shape invariance: output dimensions match specification

  - Distance preservation: pairwise distances in input space correlate with distances in projected space

- Statistical robustness: tolerance thresholds, confidence intervals, effect sizes (Cohen's d), significance tests (Mann-Whitney U)

- Combining oracles with MRs: oracles verify single-execution invariants; MRs verify cross-execution relations; together they form a comprehensive behavioral test suite

3 Related Work

3.1 Testing Machine Learning Systems

- DeepTest (Tian et al., 2018): metamorphic testing for autonomous driving DNNs — image transformations as MRs
- DeepXplore (Pei et al., 2017): neuron coverage as test adequacy criterion; differential testing across multiple DNNs
- TensorFuzz (Odena & Goodfellow, 2019): coverage-guided fuzzing for neural networks
- Zhang et al. (2020) survey on ML testing: taxonomy of test adequacy criteria, test input generation, oracle problem solutions
- Braiek & Khomh (2020): testing ML-based systems — challenges and practices
- Key takeaway: most work focuses on image classifiers / autonomous driving; less attention to embedding models and scientific software

3.2 Metamorphic Testing for Deep Learning

- Chen et al. (1998): original metamorphic testing proposal
- Segura et al. (2016): comprehensive survey of metamorphic testing across domains
- Xie et al. (2011): metamorphic testing for ML classifiers (k-NN, Naïve Bayes) — MRs based on data permutation, addition, duplication
- Dwarakanath et al. (2018): metamorphic testing for image classifiers in industry (identifying bugs in production models)
- Murphy et al. (2008): properties for testing ML programs — additive/multiplicative MRs
- Gap: MRs specifically designed for protein language model embeddings (X-masking, AA mutation, reversal) are novel

3.3 Testing Scientific and Bioinformatics Software

- Kanewala & Bieman (2014): testing scientific software — challenges (oracle problem, floating-point, rare domain expertise overlap with testing expertise)
- Hook & Kelly (2009): mutation testing for scientific software
- Storer (2017): bridging the gap between software engineering and scientific computing
- Bioinformatics-specific: limited formal testing literature; most tools rely on example-based tests and manual validation
- biotrainer, bio-embeddings: domain tools used by biocentral — their own test practices as baseline comparison

3.4 Property-Based Testing for Data Pipelines

- Claessen & Hughes (2000): QuickCheck — property-based testing with random generation
- MacIver et al. (2019): Hypothesis — property-based testing for Python
- FIT (Facets of Inadequacy in Testing): using properties to identify gaps in test suites
- Application to data pipelines: schema validation, idempotency, commutativity of transformations
- Limited prior work on applying oracle-pattern testing to ML inference pipelines specifically

3.5 Research Gap

- No prior work specifically addresses testing PLM-based embedding services with a multi-layer test strategy
- The time-vs-coverage trade-off for property-based testing in CI for ML services is unexplored
- Metamorphic relations for protein embeddings (X-masking divergence, AA mutation sensitivity, sequence reversal) are novel contributions

- GPU-free deterministic testing infrastructure for embedding services is not addressed in existing literature
- This thesis fills the gap by combining classical SE testing (unit, integration, mutation testing) with ML-specific techniques (MRs, oracles) for a real-world production system

4 Approach

4.1 Overall Test Architecture

- Multi-layer test pyramid tailored for embedding-driven ML services
- 5 layers: unit (mocked) → integration (live server) → property/oracle (invariants) → performance (benchmarks) → metamorphic experiments (MRs at scale)
- Total: ∼180 test functions across 24 test files
- Pytest markers for selective execution: `@integration`, `@slow`, `@gpu`, `@performance`, `@property`
- CI integration: markers enable running fast tests on every commit, expensive tests nightly/on-demand
- Figure idea: test pyramid diagram with layer names, test counts, and approximate execution times

4.2 Test Infrastructure

4.2.1 Canonical Test Dataset

- 37 protein sequences in `test_sequences.fasta`, organized into 8 categories
- Categories: standard (3 typical proteins), length variants (1–400 aa), unknown token variants (7 sequences with X, B, Z, J, U, O), all 20 standard AAs, homopolymers, composition extremes, ambiguous codes, structural motifs, real-world proteins (insulin B chain, ubiquitin, GFP core)
- Pydantic-based loader (`test_dataset.py`) with category filtering: `get_test_sequences(categories=''edge_case'')`
- Design rationale: cover normal cases, boundary conditions, and biologically meaningful edge cases in a single reusable dataset

4.2.2 FixedEmbedder

- Problem: real PLM inference (ESM2, ProtT5) requires GPU and takes seconds per sequence — too slow and expensive for CI
- Solution: deterministic, GPU-free embedder that produces embeddings with correct dimensionality and reproducible values
- Algorithm:
  1. Hash the input sequence via SHA-256 to obtain a deterministic seed
  2. Seed a NumPy random number generator with the hash
  3. Generate per-amino-acid base embeddings combined with positional noise
  4. Output shape matches the target PLM: e.g., ProtT5 $\rightarrow$ 1024 dims, ESM2-T33 $\rightarrow$ 1280 dims
- Singleton registry (`FixedEmbedderRegistry`) avoids redundant instantiation
- `strict_dataset` mode: optionally validates sequences against the canonical FASTA
- Key property: same sequence $\rightarrow$ same embedding, regardless of call context, batch, or machine
- 341 lines of implementation; 36 dedicated unit tests verifying determinism, dimensions, edge cases, and embedding quality (no zeros/NaN/Inf)
- Figure idea: diagram showing SHA-256 $\rightarrow$ seed $\rightarrow$ RNG $\rightarrow$ embedding pipeline

4.2.3 Integration Test Infrastructure

- 832-line `conftest.py` providing the integration test framework
- `poll_task()`: async task polling with configurable timeout, terminal status detection, consecutive error tracking
- `FakeResponse` class: intercepts outgoing HTTP calls, generates embeddings via FixedEmbedder, writes directly to PostgreSQL — enables full end-to-end flow testing without GPU
- Monkey-patching: replaces real HTTP embedding calls with FixedEmbedder at the request level

- Session-scoped fixtures for expensive resources: DB connections, HTTP clients, sequence data
- Environment variables: `CI_SERVER_URL` (server address), `CI_EMBEDDER` (mode: `esm2_t6_8m` or `fixed`)

4.3 Unit Tests

- 63 unit tests across 4 test files
- FastAPI `TestClient` with fully mocked backends: `MockedTaskManager`, `UserManager`, `FileManager`, `EmbeddingDatabaseFactory`, Redis-based `RateLimiter`
- Tests validate request/response contracts without starting the real server
- Endpoint coverage:
  - Embeddings (13 tests): embedder listing, embed requests (valid/empty/missing), missing embeddings (valid/invalid JSON/non-dict), HDF5 upload (valid/invalid base64)
  - Predictions (8 tests): model metadata, predict requests (valid/unknown/empty models/empty seqs/too short/too long sequences)
  - Projections (6 tests): config listing, project requests (valid/unknown method/missing fields/custom config passthrough)
- FixedEmbedder unit tests (36 tests): determinism (same seq, different instances, different seeds, pooled, batch→individual consistency), dimension correctness (8 model variants), edge cases (empty/single residue/very long/unknown AA/all AAs/homopolymer/lowercase), embedding quality (no zeros/NaN/Inf, float32, reasonable magnitude, position-sensitivity)
- Design principle: test validation logic and error handling exhaustively; mock all I/O and compute

4.4 Integration Tests

- 56 integration tests across 5 test files
- Run against a live server via Docker Compose (`docker-compose.dev.yml`)
- Use `httpx` with retry logic for resilience against transient failures

- Endpoint flow tests:
  - Embedding flow (7 tests): embedder listing, rejection flows, full embed → poll → cache verification, multi-embedder
  - Prediction flow (8 tests): metadata structure/consistency, rejection flows, full predict task completion
  - Projection flow (5 tests): config retrieval, rejection flows, PCA/UMAP task completion with cache verification
  - Training/inference flow (16 tests): config options (classification/regression), config verify, training start (2 protocols + error cases), inference, model files, **full train** → **inference flow** with prediction verification, train/val split
- Triton standalone tests (20 tests): all embedding models (ProtT5, ESM2-T33, ESM2-T36) pooled + per-residue; all prediction models (SecondaryStructure, Conservation, BindEmbed, TMbed, SETH, LightAttention, VespaG); model availability checks. Requires Docker with 24 GB+ RAM.
- These tests are the **main contribution**: they validate real end-to-end system behavior including database caching, task queuing, and result retrieval

4.5 Property-Based Oracle Tests

- 21 oracle tests across 3 test files
- Oracle pattern: reusable class with `check()` method returning structured result (pass/fail + metric value)
- Statistical utilities (`statistics.py`): `compute_confidence_interval` (t-distribution), `compute_cohens_d`, `derive_significance_threshold` (k-sigma), `run_with_replicates`, `mann_whitney_test`
- Metric utilities (`embedding_metrics.py`): cosine distance, L2 distance, CSV report generation
- Specific oracles and their tolerances:
  - `BatchInvarianceOracle`: embedding identical regardless of batch composition (cosine $\leq 10^{-6}$)
  - `PredictionDeterminismOracle`: same input → bit-identical output for ONNX models (exact match)

  - `OutputValidityOracle`: probabilities sum to 1, values in $[0, 1]$ (classification) or finite (regression); tolerance 10^{-5}
  - `ShapeInvarianceOracle`: output shape = (seq_len, n_classes) (exact)
  - `ProjectionDeterminismOracle`: PCA bit-identical; UMAP Procrustes distance ≤ 0.3
  - `DimensionalityOracle`: output matches requested dimensions (exact)
  - `ProjectionValueValidityOracle`: no NaN/Inf, finite values
  - `DistancePreservationOracle`: Spearman correlation ≥ 0.3 between input and projected distances
- Models tested directly via ONNX (bypassing server): ProtT5SecondaryStructure (3 classes), BindEmbed (3 classes), TMbed (7 classes), Seth (regression)

4.6 Metamorphic Experiments

- 8 experiments implemented as pytest scripts, producing CSV reports in `tests/reports/`
- All use ESM2-T6-8M as the primary model; one-hot encoding as baseline
- UniRef50-scale validation: 1,750 real protein sequences across 7 length bins (50–1000 aa), fetched via UniRef50 REST API

4.6.1 Idempotency

- Same sequence embedded multiple times must return identical results
- Tolerance: cosine distance $\leq 10^{-5}$ (allows GPU non-determinism)
- 10 sequences $\times$ 4 repeats

4.6.2 Batch Invariance

- Embedding invariant to batch composition: embed(A, B) vs. embed(A) alone
- Transformer self-attention operates within single sequences, not across batch — batch composition must be invisible
- Batch sizes: 2, 5, 10, 20; target sequence at random positions
- Pooled variant: cosine ≤ 0.01; per-residue variant: max absolute difference $\leq 10^{-5}$

4.6.3 Progressive X-Masking

- MR: progressively replace amino acids with X (unknown token) at 14 masking ratios (0–100%)
- Measure cosine distance from original embedding as a function of masking ratio
- Questions: Is divergence monotonically increasing? At what ratio r^* does cosine distance exceed 0.1?
- Progressive (superset) vs. random (independent) masking variants
- 4 variants: basic (4 sequences $\times$ 5 runs), UniRef50 scale (250 seqs/bin $\times$ 7 bins $\times$ 2 runs), one-hot baseline (4 seqs), one-hot UniRef50
- Monotonicity violations allowed $< 30\%$
- Figure idea: cosine similarity vs. masking ratio curve, ESM2 vs. one-hot vs. random baseline

4.6.4 Reversed Sequence

- MR: reverse amino acid order and compare embeddings
- ESM2 expected to be order-sensitive (positional encoding) $\rightarrow$ cosine distance > 0.001
- One-hot baseline expected to be order-insensitive $\rightarrow$ cosine distance $< 10^{-6}$
- Double-reversal identity check: reverse(reverse(seq)) $=$ seq $\rightarrow$ cosine $\leq 10^{-5}$
- Skips palindromic and repetitive sequences

4.6.5 Amino Acid Mutation Sensitivity

- MR: replace residues with each of the 20 standard amino acids at 6 masking ratios (0–100%)
- Questions: Which AA substitutions cause largest embedding change? Does sensitivity correlate with physicochemical properties (charged, hydrophobic, polar)?
- Comparison to X-masking: structured substitution vs. unknown token

- Basic variant: 4 seqs $\times$ 20 AAs $\times$ 3 runs; UniRef50 variant: 250 seqs/bin $\times$ 6 representative AAs (A, D, G, K, L, W)
- Figure ideas: heatmap (AA $\times$ masking ratio), line overlay (all 20 AAs), bar ranking by physicochemical group

4.6.6 Random Baseline

- X-masking on randomly generated (non-biological) sequences: uniform sampling from 20 standard AAs
- 7 length bins (50–1000 aa) $\times$ 50 random sequences $\times$ 2 runs
- Purpose: determine whether X-masking divergence profile differs for "meaningless" vs. real protein sequences

4.6.7 Projection Determinism

- PCA (2D, 3D): expect bit-identical coordinates across runs
- UMAP, t-SNE: stochastic $\rightarrow$ check structural similarity via Procrustes distance (≤ 0.3)

4.6.8 Training Determinism

- Same sequences + same model + same seed $\rightarrow$ identical embeddings across 3 repeated training preparations
- 8 sequences with binary labels; tolerance: cosine $\leq 10^{-5}$

4.7 Performance Tests

- 23 performance tests across 3 test files, using real ESM2-T6-8M model
- Throughput tests (10 tests): latency at 4 sequence lengths (10/15/211/400 aa), canonical dataset throughput, batch throughput, pooled embedding latency, ESM2 vs. FixedEmbedder comparison, scaling report with JSON output
- Memory tests (8 tests): leak detection (repeated single/batch with GC), footprint verification (`nbytes == seq_len × dim × 4`), pooled vs. per-residue memory ($< 10\%$ difference), memory-per-dimension ratio, batch memory estimation

- Scaling tests (5 tests): O(n) linear scaling with sequence length (ratio $< 3\times$), linear scaling with batch size, sequential vs. batch comparison
- Purpose: establish baseline performance characteristics and detect regressions

4.8 Mutation Testing

- Tool: mutmut (Python mutation testing framework)
- Scope: mutates `biocentral_server/` source code; runs only unit tests (excludes slow/integration/performance)
- Mutation operators: arithmetic, boundary, negation, statement deletion, constant replacement
- Mutation score = killed mutants / total mutants
- Purpose: measure how well the unit test suite detects injected faults — complements code coverage
- Configuration in `tests/setup.cfg`

5 Evaluation

5.1 Experimental Setup

- Hardware: specify CPU, GPU (model, VRAM), RAM
- Software: Python version, PyTorch version, ONNX Runtime version, Docker version, Triton Inference Server version
- Primary PLM: ESM2-T6-8M (8 million parameters) — chosen for fast iteration while retaining transformer behavior
- Baseline encodings: one-hot encoding, BLOSUM62
- Docker Compose configuration: server, 4 workers, Redis, PostgreSQL, SeaweedFS, Triton
- All experiments run on the same machine for fair comparison; report wall-clock times

5.2 Test Coverage and Mutation Testing Results

- Total test count: $\sim$180 test functions across 24 files
- Code coverage (line coverage, branch coverage) for unit tests — report numbers with `pytest-cov`
- Mutation testing results:
  - Total mutants generated by mutmut
  - Killed vs. survived mutants
  - Mutation score (%)
  - Which types of mutants survived — what does this reveal about test suite weaknesses?
  - Comparison: mutation score vs. line coverage — are they correlated?

- Table idea: mutation score breakdown by module (embeddings, predictions, projections, etc.)

5.3 CI Execution Time Analysis

- Wall-clock execution time per test category:
  - Unit tests (with mocks): expected ∼seconds
  - Integration tests (with FixedEmbedder): expected ∼minutes
  - Integration tests (with real ESM2): expected ∼longer
  - Property/oracle tests: time per oracle
  - Performance tests: time per benchmark
  - Metamorphic experiments (basic): time for 4-sequence variants
  - Metamorphic experiments (UniRef50 scale): time for 1,750-sequence variants
- **Key analysis**: plot fault-detection value (mutation score contribution, bug-finding capability) against CI time cost
- Trade-off curve: diminishing returns — at what point does adding more property tests stop being worth the CI time?
- FixedEmbedder speedup: how much faster is CI with fixed vs. real embedder? What coverage is sacrificed?
- Recommendation: optimal test portfolio for different CI budgets (e.g., per-commit: unit + fixed integration; nightly: full suite; weekly: UniRef50 experiments)
- Figure idea: stacked bar chart of CI time per layer; scatter plot of time vs. coverage contribution

5.4 Metamorphic Experiment Results

5.4.1 X-Masking Results

- Cosine divergence profiles: plot cosine distance vs. masking ratio for ESM2
- Critical ratio r^*: at what masking percentage does cosine distance exceed 0.1?
- Monotonicity analysis: percentage of sequences with non-monotonic divergence; characterize violations

- ESM2 vs. one-hot comparison: one-hot should show linear divergence (bag-of-features); ESM2 shows non-linear (contextual model)
- ESM2 vs. random baseline: do random sequences diverge differently than real proteins?
- UniRef50 length-bin breakdown: does divergence profile depend on sequence length?
- Figure ideas: cosine similarity curve (ESM2 + one-hot + random), per-bin breakdown, L2 distance dual panel

5.4.2 Reversed Sequence Results

- Distribution of cosine distances for ESM2 reversed sequences — confirm order-sensitivity
- One-hot baseline: confirm near-zero distance (order-insensitive, as expected)
- Double-reversal identity: confirm recovery within tolerance
- Interpretation: positional encoding in ESM2 captures sequence directionality

5.4.3 AA Mutation Sensitivity Results

- Heatmap: amino acid $\times$ masking ratio showing cosine distance
- Sensitivity ranking: which AAs cause largest/smallest embedding changes at $\sim$10% mutation?
- Physicochemical grouping: charged (D, E, K, R) vs. hydrophobic (A, V, L, I) vs. polar (S, T, N, Q) vs. special (G, P, C, W)
- Comparison to X-masking: is replacing with a real AA more or less disruptive than replacing with X?
- UniRef50 scale: do sensitivity patterns hold across diverse real proteins?
- Figure ideas: heatmap, bar chart by physicochemical group, mutation vs. X-masking overlay

5.5 Oracle Test Findings

- Batch invariance: report cosine distances (expect $\ll 10^{-6}$); any violations?
- Prediction determinism: confirm bit-identical ONNX outputs across runs
- Output validity: any probabilities outside $[0, 1]$ or not summing to 1? Any NaN/Inf?
- Shape invariance: any dimension mismatches?
- Projection distance preservation: Spearman correlation values for PCA, UMAP, t-SNE
- Projection determinism: PCA exact; UMAP/t-SNE Procrustes distances
- Table idea: oracle name, metric, threshold, observed value, pass/fail

5.6 Performance Results

- Latency by sequence length: plot latency vs. sequence length (10, 15, 211, 400 aa)
- Throughput: sequences per second for batch processing
- Memory: footprint per embedding, leak detection results
- $O(n)$ scaling: confirm linear scaling with sequence length (ratio $< 3\times$)
- Batch efficiency: batch vs. sequential processing speedup
- ESM2 vs. FixedEmbedder: orders-of-magnitude speed difference quantified
- Table/figure idea: latency table, memory bar chart, scaling line plot

5.7 Discussion of Results

- **Answering the RQ**: What is the trade-off between CI execution time and test coverage?
  - Unit tests: fastest, catch validation/contract bugs, limited for ML-specific issues
  - Integration tests (FixedEmbedder): moderate time, catch real flow bugs without GPU — best bang-for-buck

  - Oracle/property tests: moderate time, catch invariant violations that unit tests miss
  - Metamorphic experiments: slow (especially UniRef50), provide unique behavioral insights but diminishing returns for CI
  - Performance tests: moderate time, essential for regression detection

- Recommended CI strategy: per-commit (unit + fixed integration), nightly (+ oracles + performance), weekly/release (+ metamorphic + real GPU)
- FixedEmbedder as enabler: makes the pyramid viable by decoupling CI from GPU availability
- Surprising findings (if any): unexpected oracle failures, non-monotonic masking behavior, etc.

6 Discussion

6.1 Trade-Off Summary

- Restate the RQ and summarize the answer with concrete numbers from Chapter 5
- Optimal allocation of CI budget across test layers
- The FixedEmbedder as a key enabler: percentage of integration tests that can run GPU-free; what fraction of real bugs does it catch vs. miss?
- Diminishing returns: at what point does adding more metamorphic tests stop improving fault detection?
- Cost of false security: what types of bugs can only be caught with real GPU inference?

6.2 Generalizability

- Beyond biocentral: applicability to other PLM-based services (e.g., ColabFold, bio-embeddings servers)
- Beyond protein embeddings: applicability to NLP embedding services (sentence transformers, BERT-based APIs)
- Beyond embeddings: applicability to other ML inference services (image classifiers, recommendation systems)
- Which parts of the approach are domain-specific (canonical protein dataset, AA mutation MRs) vs. generalizable (test pyramid, FixedEmbedder pattern, oracle framework)?
- Multi-service testing: the approach handles complex Docker deployments (server + workers + GPU + DB) — relevant for any microservice-based ML system

6.3 Limitations

- Single PLM for experiments: ESM2-T6-8M is the smallest ESM2 variant; larger models (650M, 3B) or different architectures (ProtT5) may exhibit different behavior
- No adversarial testing: the metamorphic experiments use structured transformations, not adversarial inputs designed to fool models
- FixedEmbedder expressiveness: SHA-256-based embeddings do not capture real biological properties — bugs that require biologically meaningful embeddings will be missed
- No end-user study: the test suite was developed by the author; no external developer usability evaluation
- Limited mutation testing scope: mutmut only targets unit tests; integration/property tests not evaluated for mutation score
- Hardware-specific timings: execution times depend on specific CPU/GPU; relative ratios are more meaningful than absolutes

6.4 Threats to Validity

- Internal validity:
  - GPU non-determinism tolerance thresholds were chosen empirically — different thresholds could change pass/fail outcomes
  - FixedEmbedder monkey-patching may mask real HTTP/serialization bugs
  - Mutation testing results depend on mutmut's mutation operators — different tools may produce different scores
- External validity:
  - Results specific to biocentral's architecture (FastAPI + Redis + Triton) — other frameworks may have different failure modes
  - ESM2-T6-8M behavior may not generalize to larger or differently trained models
  - UniRef50 sequences are biased toward well-studied proteins — rare/novel proteins underrepresented
- Construct validity:

- "Test coverage" measured via mutation score and code coverage — other metrics (fault detection rate in production) would be more meaningful but are unavailable
- "CI execution time" measured on a single machine — cloud CI environments may differ

7 Conclusion

7.1 Summary of Contributions

- Multi-layer test pyramid for embedding-driven ML services: unit → integration → property/oracle → performance → metamorphic experiments
- FixedEmbedder: deterministic, GPU-free test infrastructure enabling fast CI for PLM-based services
- 8 metamorphic experiments for PLM embeddings: X-masking divergence, reversed sequence sensitivity, AA mutation sensitivity, with validation at UniRef50 scale (1,750 real proteins)
- Oracle test pattern with statistical utilities for robust invariant checking under numerical noise
- Mutation testing evaluation quantifying unit test fault-detection capability
- Quantitative analysis of the time-vs-coverage trade-off: concrete recommendations for CI test portfolios

7.2 Key Findings

- Summarize the most important result for the RQ (e.g., "Integration tests with FixedEmbedder provide X% of fault detection at Y% of the CI time of full GPU tests")
- Metamorphic experiments reveal: [insert key behavioral findings about ESM2, e.g., critical masking ratio, sensitivity ranking]
- Oracle tests confirm: [insert which invariants hold/fail, e.g., batch invariance holds perfectly, projection distance preservation varies by method]
- Mutation testing reveals: [insert mutation score and what it implies about test adequacy]

7.3 Future Work

- Extend metamorphic experiments to larger PLMs: ESM2-650M, ESM2-3B, ProtT5-XL — do behavioral properties change with model scale?
- Adversarial testing: design inputs that maximize embedding instability or prediction errors
- Automated MR discovery: use search-based techniques to find new metamorphic relations that maximize fault detection
- Coverage-guided property testing: combine mutation testing feedback with property test generation to target untested code
- Cross-platform evaluation: apply the test strategy to other ML serving frameworks (TensorFlow Serving, TorchServe, BentoML)
- Production monitoring: extend oracle tests into runtime monitors that flag anomalous embeddings in production
- Developer study: evaluate the test suite's usability with external developers contributing to biocentral

Abbreviations

PLM Protein Language Model

CI Continuous Integration

API Application Programming Interface

GPU Graphics Processing Unit

RQ Research Question

ML Machine Learning

List of Figures

List of Tables

Bibliography

[Ber+25] J. H. Bernardo, D. A. da Costa, F. R. Cogo, S. Q. de Medeiros, and U. Kulesza. *Continuous Integration Practices in Machine Learning Projects: The Practitioners' Perspective*. 2025. arXiv: 2502.17378 [cs.SE].

[Cha+23] J. Chandrasekaran, T. Cody, N. McCarthy, E. Lanus, and L. Freeman. *Test & Evaluation Best Practices for Machine Learning-Enabled Systems*. 2023. arXiv: 2310.06800 [cs.SE].

[Eln+21] A. Elnaggar, M. Heinzinger, C. Dallago, G. Rihawi, Y. Wang, L. Jones, T. Gibbs, T. Feher, C. Angerer, M. Steinegger, D. Bhowmik, and B. Rost. *ProtTrans: Towards Cracking the Language of Life's Code Through Self-Supervised Deep Learning and High Performance Computing*. 2021. arXiv: 2007.06225 [cs.LG].

[Fra+25] S. Franz, T. Olenyi, P. Schloetermann, A. Smaoui, L. F. Jimenez-Soto, and B. Rost. *biocentral: embedding-based protein predictions*. Nov. 2025. DOI: 10.1101/2025.11.21.689449.

[Rzi+25] D. E. Rzig, A. Houerbi, R. G. Chavan, and F. Hassan. *Empirical Analysis on CI/CD Pipeline Evolution in Machine Learning Projects*. 2025. arXiv: 2403.12199 [cs.SE].